# Thinking Token Measurement in Claude Code + Opus 4.6

Slavko Klincov, klincov.it

April 2026, Revision 3

## Thinking Token Measurement in Claude Code + Opus 4.6

Empirische Analyse, April 2026. Basierend auf 13 kontrollierten Experimenten, 26,484 API-Calls Tageszeit-Analyse, und Transcript-Analyse. Rev. 3.

---

## Teil I: Messmethode

### 1. Architektur: Wer steuert was

Claude Code hat **drei Steuerungsebenen** fuer Thinking, plus ein verstecktes viertes Limit:

```
Ebene 1: settings.json (env vars)     → Claude Code CLI → API budget_tokens
Ebene 2: settings.json (effortLevel)  → Claude Code CLI → API effort Parameter
Ebene 3: ~/.claude/CLAUDE.md          → In Kontext injected → Beeinflusst
Modellverhalten
Ebene 4: Claude Code Binary           → Hardcoded max_tokens (NICHT
konfigurierbar)
```

#### Ebene 1: budget_tokens (settings.json env)

```
"CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING": "1",
"MAX_THINKING_TOKENS": "128000"
```

- `MAX_THINKING_TOKENS` setzt `budget_tokens` im API-Request: die **Obergrenze** fuer Thinking-Tokens pro Call
- `DISABLE_ADAPTIVE_THINKING` erzwingt den manuellen Modus (feste Obergrenze statt dynamischer Zuweisung)
- Auf Opus 4.6 ist der manuelle Modus (`budget_tokens`) **deprecated**, funktioniert aber noch
- Ein Vergleichsexperiment (Abschnitt 5) zeigt, dass der Modus keinen messbaren Einfluss auf die Thinking-Trigger-Rate hat

#### Ebene 2: effort (settings.json)

```
"effortLevel": "max"
```

- `max` ist das hoechste Level (nur auf Opus 4.6)
- Beeinflusst, wie viel Aufwand Claude in Reasoning investiert
- `/effort max` im Chat setzt dies fuer die aktuelle Session

### Ebene 3: CLAUDE.md (Kontext-Anweisungen)

```
# Output
- Minimize output tokens. No verbose narration around tool calls.
```

- CLAUDE.md steuert **Verhalten**, nicht API-Parameter
- **Potentieller Konflikt**: “Minimize output tokens” koennte Thinking unterdruecken
- Prompt-Anweisungen wie “ultrathink” wirken auf derselben Ebene und koennen CLAUDE.md situativ ueberwiegen

### Ebene 4: max_tokens (versteckt, hardcoded)

Aus dem Claude Code Binary extrahiert:

| Kontext | max_tokens |
|---|---|
| Ohne Thinking | 16,000 |
| Mit Thinking | **64,000** |

`max_tokens` begrenzt den **gesamten** Output (Thinking + sichtbare Antwort). Die API erzwingt: `budget_tokens < max_tokens`. Effektiv:

```
Unser MAX_THINKING_TOKENS = 128,000 (budget_tokens, Client-Ceiling)
Claude Code max_tokens     =  64,000 (hardcoded, effektives Limit)
→ Thinking + Antwort zusammen nie mehr als 64k pro Call
```

`CLAUDE_CODE_MAX_OUTPUT_TOKENS` als Env-Var existiert, ist aber standardmaessig nicht gesetzt. Ob sie max_tokens ueberschreibt, ist nicht verifiziert.

### Zusammenspiel

```
max_tokens = 64,000          ← Harte Obergrenze total (hardcoded)
budget_tokens = 128,000      ← Thinking-Ceiling (ignoriert weil > max_tokens)
effortLevel = max            ← Tendenz zu mehr Thinking (API)
CLAUDE.md = "Minimize"       ← Tendenz zu weniger Output (Kontext)
Server-Allokation = variabel ← Anthropic throttelt serverseitig (Abschnitt 7)
```

---

## 2. Herleitung: Vom Fehler zur Erkenntnis

Die Ergebnisse sind nicht linear entstanden. Der Weg enthielt mehrere Fehlschluesse, und jede Korrektur fuehrte zu einer praeziseren Messmethode.

### Schritt 1: Das Ausgangsproblem

Die urspruengliche Statusline verglich `context_window.total_output_tokens` (kumulativ ueber die Session) gegen `MAX_THINKING_TOKENS=64000` (Per-Turn-Limit). Ein Session-Zaehler gegen ein Per-Call-Budget: konzeptuell falsch.

### Schritt 2: Erster Ansatz mit chars/4 und “Phantom-Thinking”

Die erste Thinking-Formel nutzte den gaengigen Divisor 4.0 (chars/token):

```
thinking_est = output_tokens - (visible_chars / 4)
```

Ergebnis: Non-Thinking-Calls zeigten dreistellige “Phantom-Thinking-Tokens” (CAP Theorem: 557, TCP vs UDP: 171), obwohl Thinking definitiv AUS war. Wir haben das als erwartetes Rauschen akzeptiert statt als Kalibrierungsfehler.

**Schritt 3: Der “Summary”-Fehlschluss**

Drei Research-Agents untersuchten die Anthropic API. Ihre Schlussfolgerung: “output_tokens enthaelt nur Summary-Tokens, echtes Thinking ist 10-17x hoeher.” **Wir haben das unkritisch uebernommen** und den Balken als `(summary, est.)` gelabelt. Die Agents hatten eine Teilwahrheit geliefert: der sichtbare Thinking-**Text** IST eine Summary. Aber `usage.output_tokens` ist es nicht.

**Schritt 4: Der Widerspruch**

Die Frage “das bedeutet also keine Zahlen?” fuehrte zur entscheidenden Einsicht:

> Wenn `output_tokens` nur Summaries enthaelt, warum zeigen Non-Thinking-Calls “Phantom-Thinking”? Diese Calls HABEN kein Thinking, also muss `output_tokens` dort den sichtbaren Text-Tokens entsprechen.

Daraus folgt: der chars-per-token Divisor laesst sich empirisch bestimmen.

**Schritt 5: Kalibrierung und Beweis**

Non-Thinking-Baselines ergeben ~2.7 chars/token (nicht 4.0). Mit dem kalibrierten Divisor konvergieren Non-Thinking-Cases auf ~0 (±26 Tokens). Das beweist: `usage.output_tokens` enthaelt volle Thinking-Tokens, nicht Summaries.

**Schritt 6: Peer-Review Korrekturen**

1. “10-17x widerlegt” → “aufgeloest” (beide Messpunkte korrekt, verschiedene Methoden)
2. Thinking-Block ≠ echtes Reasoning (Summary oder leer)
3. Deprecated-Mode-Hypothese empirisch widerlegt (2/6 in beiden Modi)

**Schritt 7: Turn-Aggregation**

Die Statusline zeigte Thinking nur fuer den **letzten** API-Call eines Turns. Schweres Thinking passiert aber in fruehen Calls (Planung/Reasoning), waehrend spaetere Calls Tool-Use sind. Vor dem Fix: ~1.1k angezeigt. Nach dem Fix (Turn-Aggregation): ~6.1k, weil jetzt alle Calls seit der letzten User-Message zusammengezaehlt werden.

Dabei entdeckt: Das Transcript nutzt `"type":"user"` fuer User-Messages, nicht `"type":"human"`. Tool-Results haben ebenfalls `type:"user"`, aber mit Array-Content statt String. Die Turn-Boundary-Erkennung filtert jetzt korrekt auf String-Content.

---

## 3. Ergebnis: Zwei Messpunkte, ein Missverstaendnis

### Der "10-17x Undercount"-Claim: aufgeloest, nicht widerlegt

| Messpunkt | Was er misst | Korrektheit |
|---|---|---|
| `usage.output_tokens` (API-Response) | **Volle** Thinking-Tokens + sichtbarer Output | Korrekt, vollstaendig |
| Content-Block-Text (thinking + text Felder) | **Summary/redacted** Thinking + sichtbarer Output | Massiv unterzaehlt |

Anthropic-Dokumentation (Extended Thinking): > "Output tokens (billed): The original thinking tokens that Claude generated internally." > "Output tokens (visible): The summarized thinking tokens you see in the response."

Beide Seiten haben recht. `usage.output_tokens` enthaelt die vollen Tokens. Tools die aus Content-Bloecken zaehlen, unterzaehlen tatsaechlich massiv (9.2x in unserem Test).

**Kalibrations-Beweis**

Bei Prompts OHNE Thinking ist `usage.output_tokens` = sichtbare Text-Tokens. Daraus: chars/token = 2.55-2.71, Mittel ~2.7.

| Prompt | Thinking? | output_tokens | text_chars | visible_tok (chars/2.7) | thinking_est |
|---|---|---|---|---|---|
| TCP vs UDP | OFF | 474 | 1,209 | 448 | **26 ≈ 0** ✓ |
| CAP Theorem | OFF | 1,728 | 4,683 | 1,734 | **-6 ≈ 0** ✓ |
| Fixed-point Beweis | ON | 2,927 | 6,012 | 2,227 | **700** |
| Flusspuzzle+Fuchs | ON | 5,750 | 1,691 | 626 | **5,124** |

Die Formel `thinking_tokens ≈ output_tokens - (visible_chars / 2.7)` liefert **reale** Thinking-Token-Werte.

---

## 4. Thinking-Detection

### Definitiver ON/OFF-Indikator

Das Transcript speichert pro API-Call Content-Blocks mit Typ-Annotation. Wenn Thinking aktiv war, existiert ein Block mit `"type": "thinking"`. Der Text darin ist bei Claude 4.x eine Summary oder komplett leer (redacted), aber die **Praesenz des Blocks** ist zuverlaessig.

### Die 4 Saeulen + Signature

| # | Metrik | Quelle | Zuverlaessigkeit | Bedeutung |
|---|---|---|---|---|
| 1 | `output_tokens` | usage-Objekt | 100% | Volle Output-Tokens inkl. Thinking |
| 2 | `text_chars` | text/ tool_use Blocks | 100% | Sichtbarer Output (ohne Thinking) |
| 3 | `cache_read_input_tokens` | usage-Objekt | 100% | Konversationshistorie |
| 4 | **`has_thinking`** | content_types | **100%** | **Definitiver ON/OFF-Indikator** |
| 5 | `signature` (optional) | thinking Block | Sekundaer | Laenge korreliert mit Thinking-Tiefe (r=0.68) |

Issue #42796 berichtet r=0.971 Korrelation zwischen Signature-Laenge und Thinking-Textlaenge (7,146 Blocks mit sichtbarem Text, pre-Redaction). Wir konnten das **nicht reproduzieren**: 99.2% unserer 393 Thinking-Blocks sind redacted. Die Signature ist ein ergaenzender Indikator, kein Ersatz fuer unsere Formel.

### Intensitaets-Klassifikation

Basierend auf `output_tokens / visible_tokens`:

| Ratio | Intensitaet | Bedeutung |
|---|---|---|
| < 2.0 | Kein/minimal | Kein signifikantes Thinking |
| 2.0 - 5.0 | Light | Kurzes Reasoning |
| > 5.0 | Heavy | Intensives Deep Thinking |

# Teil II: Erkenntnisse

## 5. Experiment: Adaptive vs Manual Thinking

Gleiche 6 Prompts, Manual (`DISABLE_ADAPTIVE_THINKING=1`) vs Adaptive (`=0`), `effortLevel=max`.

| Prompt | Manual: Think? | Manual: OUT | Adaptive: Think? | Adaptive: OUT |
|---|---|---|---|---|
| 2+2 trivial | OFF | 6 | OFF | 6 |
| 3 languages | OFF | 17 | OFF | 17 |
| TCP vs UDP | OFF | 474 | OFF | 402 |
| CAP theorem deep | OFF | 1,728 | OFF | 1,835 |
| Fixed-point proof | **ON** | 2,927 | **ON** | 3,165 |
| River puzzle+fox | **ON** | 5,750 | **ON** | 4,223 |

**2/6 Trigger in beiden Modi. Identisch.** Der Modus beeinflusst die Thinking-Aktivierung nicht.

---

## 6. Maximum-Thinking-Test

Ein gezielt maximaler Prompt: 5-teiliger formaler Beweis eines verteilten Lock-Protokolls mit Byzantine-Fehleranalyse, adversarialen Szenarien, TLA+-Spezifikation, und Vergleich gegen PBFT/Raft/Viewstamped Replication.

| Metrik | Wert |
|---|---|
| Total output_tokens | 35,026 (neuer Rekord) |
| Geschaetzte Thinking-Tokens | ~22,334 |
| Thinking-Budget-Nutzung | 17.4% von 128k |
| Sichtbare Antwort | ~34,800 chars |

Selbst der haerteste Prompt nutzte nur 17% des Budgets. Im Kontext von Abschnitt 7 (Server-Throttle) und Abschnitt 1 (max_tokens=64k) stellt sich die Frage ob Claude freiwillig aufhoerte oder begrenzt wurde.

---

## 7. Server-seitige Thinking-Allokation

Eine unabhaengige Analyse (GitHub Issue anthropics/claude-code#42796, 17,871 Thinking-Blocks, 234,760 Tool-Calls) deckt auf, dass Anthropic Thinking-Budgets **server-seitig allokiert und throttelt**.

### Kernbefunde aus #42796

**Thinking-Tiefe sank um 73%** zwischen Januar und Maerz 2026:

| Zeitraum | Median Thinking (chars) | vs Baseline |
|---|---|---|
| Jan 30 - Feb 8 (Baseline) | ~2,200 | - |
| Ende Februar | ~720 | –67% |
| Maerz (nach Redaction) | ~600 | –73% |

Die Reduktion begann **vor** der Thinking-Redaction, also serverseitig, unabhaengig von Client-Config.

**Qualitaets-Impact**: Read:Edit Ratio 6.6→2.0 (-70%), 33% Edits ohne vorheriges Lesen, Stop-Hook-Verletzungen 0→173, User-Interrupts 12x.

### Drei separate Ceilings

Unsere Analyse identifiziert drei unabhaengige Limits die Thinking begrenzen:

| Ceiling | Wert | Quelle | Kontrollierbar? |
|---|---|---|---|
| `budget_tokens` (MAX_THINKING_TOKENS) | 128,000 | Client-Config | Ja |
| `max_tokens` (hard-coded) | 64,000 | Claude Code Binary | Nein (CLAUDE_CODE_MAX_OUTPUT_TOKENS unklar) |
| Server-Allokation | variabel, load-abhaengig | Anthropic Infrastruktur | Nein |

Das effektive Limit ist das Minimum aller drei. In der Praxis dominiert die Server-Allokation.

### Implikation

Unsere chars/2.7 Formel misst korrekt was **allokiert** wurde. Was Claude **gebraucht haette**, koennen wir nicht messen.

---

## 8. Tageszeit-Analyse

### Methode

Analyse von **26,484 API-Calls** aus bestehenden Transcript-Dateien (`~/.claude/projects/`), gruppiert nach Stunde (lokale Zeit MESZ). Keine neuen Test-Runs noetig, die Daten lagen bereits vor.

### Ergebnisse

| Stunde (MESZ) | PST | API-Calls | Thinking-Calls | Rate | Avg Output |
|---|---|---|---|---|---|
| 06:00 | 21:00 (US Off-Peak) | 1,594 | 278 | **17%** | 1,221 |
| 12:00 | 03:00 (US Nacht) | 1,954 | 295 | 15% | 554 |
| 13:00 | 04:00 (US Nacht) | 2,010 | 329 | 16% | 647 |
| 14:00 | 05:00 (US frueh) | 2,718 | 364 | 13% | 612 |
| 17:00 | 08:00 (US Morgen) | 901 | 74 | 8% | 249 |
| 18:00 | 09:00 (US Peak) | 278 | 15 | **5%** | 254 |

### Interpretation

Thinking-Rate schwankt von **5% (18:00 MESZ, US-Peak)** bis **17% (06:00 MESZ, US-Off-Peak)**. Das Muster ist konsistent mit server-seitiger, load-abhaengiger Thinking-Allokation und bestaetigt die Befunde aus Issue #42796 mit eigenen Daten.

**Caveat**: Diese Analyse hat einen Confounding Factor. Der User arbeitet moeglicherweise unterschiedlich zu verschiedenen Tageszeiten (einfachere Aufgaben abends, komplexere morgens). Issue #42796 kontrollierte dafuer mit identischen Prompts zu verschiedenen Zeiten. Unsere Analyse nutzt historische Daten ohne Prompt-Kontrolle.

---

## 9. Zusammenfassung der Befunde

| Befund | Details |
|---|---|
| chars/token = 2.7, nicht 4.0 | Fuer Markdown/technischen Text. Fuehrt bei falscher Annahme zu ~50% Ueberschaetzung |
| Max Thinking: 22.3k (17.4%) | Selbst haertester Prompt nutzt nur 17% des 128k Budgets |
| 3 Ceilings | budget_tokens (128k, Client), max_tokens (64k, hardcoded), Server-Allokation (variabel) |
| Tageszeit-Variation: 5-17% | US-Off-Peak = mehr Thinking, US-Peak = weniger |
| Turn-Aggregation: 6x mehr | Single-Call-Messung zeigte ~1.1k, Turn-Aggregation zeigt ~6.1k |
| Adaptive = Manual | Identische 2/6 Trigger-Rate |
| Server-Throttle wahrscheinlich | #42796 dokumentiert 73% Reduktion, unsere Tageszeit-Daten bestaetigen |

---

# Teil III: Praxis

## 10. Praktische Implementierung: Statusline

Die Erkenntnisse sind in `~/.claude/statusline.sh` implementiert (3 Zeilen, turn-aggregiert):

```
5h Limit ▉▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒ 14% 2h | 7d Limit ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒ 3% Fr 02:00 | ctx 22%
last API-Call 960 | session 565.7k | cached history 224.7k
thinking: ON ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒ 0% ~243 this turn (light, 6/20 calls sig:2.9k)
```

- **Zeile 1**: Rate Limits (5h/7d) mit Countdown/Reset-Datum, Context-Window-Auslastung
- **Zeile 2**: Neuer Token-Verbrauch (ohne Cache-Reads), Session-Total, Cache-Last
- **Zeile 3**: Thinking ON/OFF (definitiv via content_types), Turn-aggregierte Thinking-Tokens, Calls-Zaehler, Signature-Laenge als sekundaerer Indikator

"this turn" = aggregiert ueber alle API-Calls seit der letzten User-Message, nicht nur der letzte Call. Datenquelle: `usage.output_tokens` aus dem JSONL-Transcript.

---

## 11. Offene Fragen

1. **Server-Throttle vs Modell-Effizienz**: Tageszeit-Variation (5-17%) deutet auf Server-Throttle. Eindeutiger Beweis wuerde kontrollierte Prompts zu verschiedenen Tageszeiten erfordern (wie in #42796).
2. **max_tokens=64k als Bottleneck?** Ob `CLAUDE_CODE_MAX_OUTPUT_TOKENS=128000` das hardcoded Limit ueberschreibt, ist nicht getestet.
3. **Subscription-Tier-Abhaengigkeit**: Pro vs Max Subscription koennten unterschiedliche Server-seitige Thinking-Budgets haben. Issue #42796 zeigt $42k geschaetzte Kosten bei $400 Subscription.
4. **"Minimize output tokens" Konflikt**: Die CLAUDE.md-Anweisung koennte Thinking unterdruecken. Experiment steht aus.
5. **print-mode vs interaktiv**: Thinking war im interaktiven Modus haeufiger aktiv als in `claude -p`. Unterschiedliche API-Parameter oder Kontext-Unterschiede moeglich.

---

## Methodik

- 13 kontrollierte Prompts: 6 Manual + 6 Adaptive + 1 Maximum-Test, jeweils `claude -p` an Opus 4.6
- 26,484 API-Calls Tageszeit-Analyse aus bestehenden Transcripts
- 393 Thinking-Blocks analysiert (Signature-Korrelation)
- Selbst-Kalibrierung via Non-Thinking-Baseline (chars/token = 2.7)
- 55/55 automatisierte Tests PASS (Statusline-Verifizierung)
- Settings: `MAX_THINKING_TOKENS=128000`, `CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING=1`, `effortLevel=max`

### Revision 3 Korrekturen

1. **max_tokens vs budget_tokens**: Zwei separate Ceilings entdeckt (64k hardcoded vs 128k Config)
2. **Server-seitiges Throttling**: Issue #42796 eingeordnet, eigene Tageszeit-Daten bestaetigen
3. **Turn-Aggregation**: Statusline zeigte nur letzten Call (Bug: "human" statt "user"), aggregiert jetzt ueber ganzen Turn
4. **Tageszeit-Analyse**: 26k Calls zeigen 5-17% Thinking-Variation nach Tageszeit
5. **Maximum-Thinking-Test**: 22.3k/17.4% mit haertestem Prompt
6. **Signature-Indikator**: r=0.68, #42796 r=0.971 nicht reproduzierbar (99.2% redacted)

---

Slavko Klincov, klincov.it, April 2026, Revision 3